

DOCUMENTACIÓN DEL PROYECTO

# k0lmenIA

Agentes de QA para Claude Code: diseño de pruebas, automatización con k0lmena e integración con Xray, QMetry y AIO Tests — todo en español.

17 agentes

Web · API · Mobile · Performance

Playwright · Cucumber · Appium · k6 · Artillery

Xray · QMetry · AIO Tests

Jira · Confluence · Figma

Suites sin tokens

QARMY · Underc0de

[github.com/underc0delabs/k0lmenIA](https://github.com/underc0delabs/k0lmenIA)

Licencia MIT

# Contenido

- 1 Qué es k0lmenIA
- 2 Arquitectura
- 3 Cómo se usa
- 4 Agentes
- 5 Requisitos
- 6 Instalación
- 7 Configuración: el archivo .env
- 8 Guía por tarea: diseño y ejecución
- 9 Automatización con k0lmena
- 10 Pruebas de performance
- 11 Gestión de pruebas: Xray, QMetry y AIO Tests
- 12 Conectores MCP
- 13 Convenciones
- 14 Estructura del repositorio
- 15 Problemas frecuentes
- 16 Extender el proyecto

INTRODUCCIÓN

# Qué es k0lmenIA

Un equipo de **agentes especializados en QA** para Claude Code, pensado para profesionales de testing manual: no hace falta programar, se trabaja conversando dentro de VS Code.

Ponés un insumo (una historia, una observación de bug, un contrato de API, o directamente una key de Jira o un link de Figma), pedís lo que necesitás en lenguaje natural y el agente que corresponde genera el resultado: análisis, planes, casos en Excel o Gherkin, datos, bugs, reportes HTML, automatización y carga en la herramienta de gestión.



### 1 • Diseño

Analizar historias, planificar, escribir casos manuales, BDD y de API, generar datos y redactar bugs.

Queda en `output/`



### 2 • Automatización

Los agentes *mapper* escriben la automatización una sola vez; después corre con `npm`, sin tokens.

Queda en `herramientas/k0lmena/`



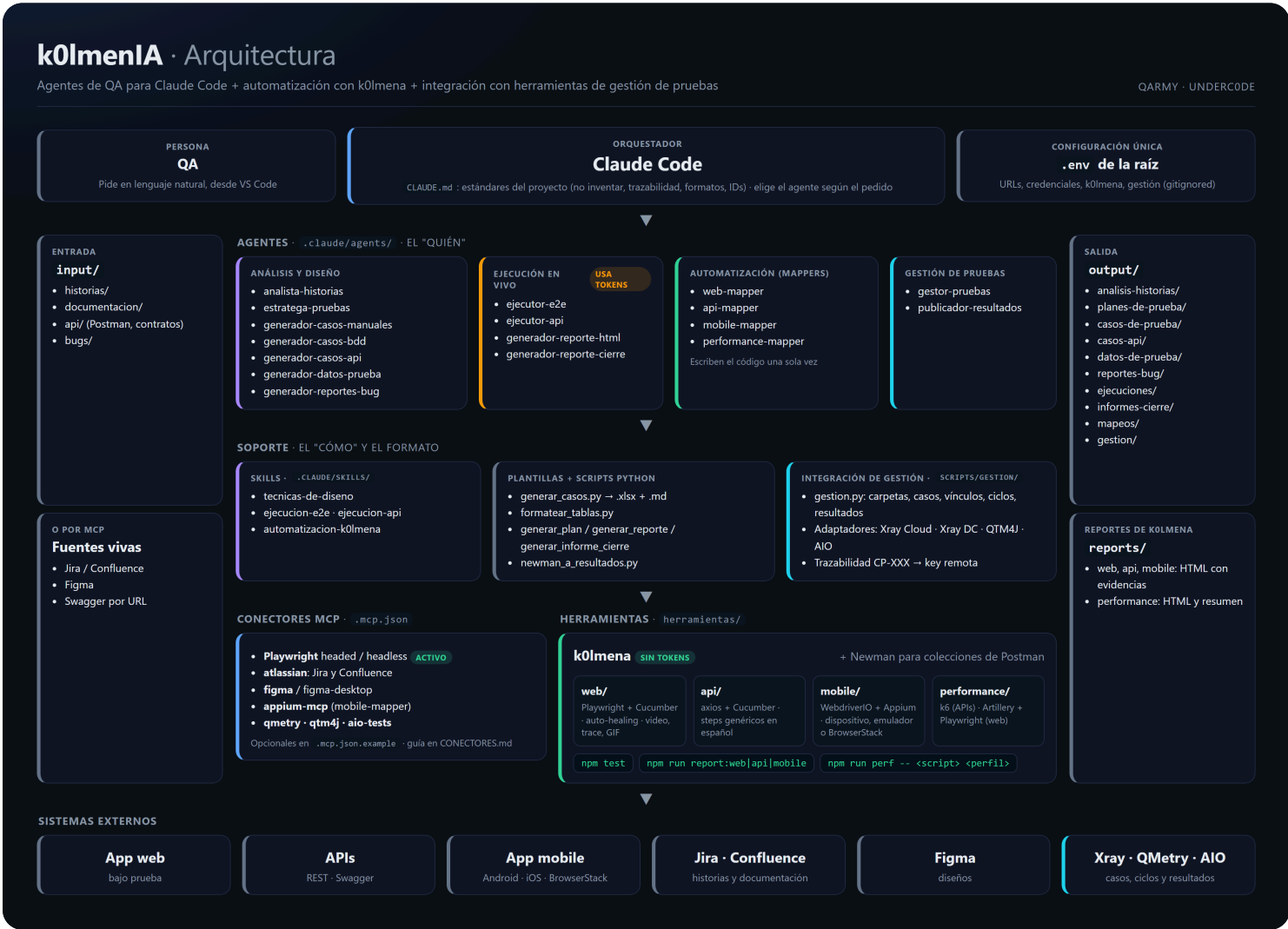
### 3 • Gestión

Casos, ciclos y resultados con evidencias en Xray, QMetry o AIO Tests, y el informe de cierre.

Tu herramienta + `output/`

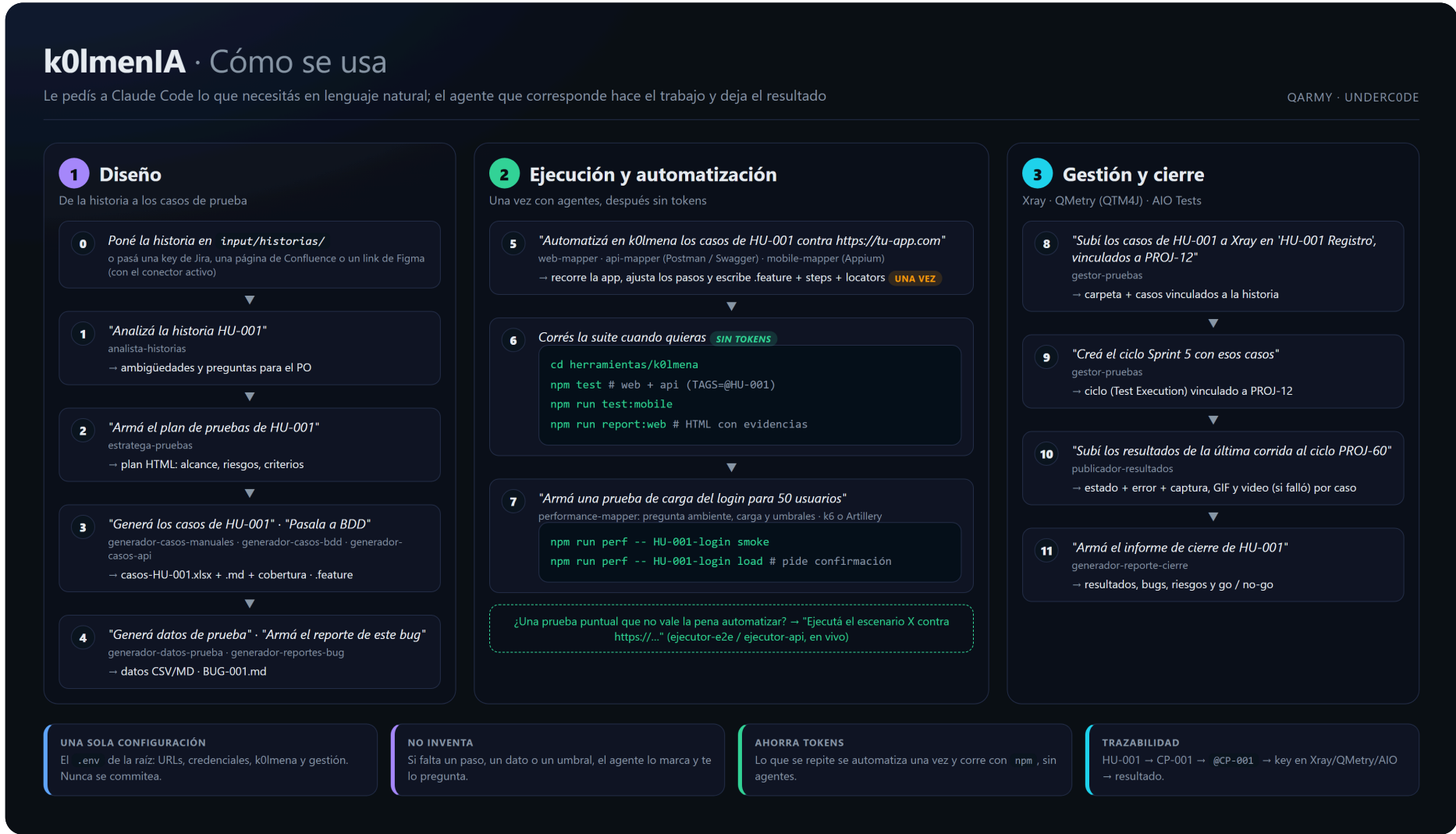
## Principios

| PRINCIPIO             | EN LA PRÁCTICA                                                                                                                                                        |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| No inventar           | Si falta un paso, un dato, una regla o un umbral, el agente lo marca y lo pregunta. Mejor un artefacto con huecos señalados que uno completo pero inventado.          |
| Respetar los formatos | Los campos, su orden y sus valores los definen las plantillas y los scripts, no el agente.                                                                            |
| Trazabilidad          | Todo referencia la historia ( <code>HU-XXX</code> ); cada escenario automatizado lleva el ID del caso ( <code>@CP-001</code> ) y la key de la herramienta de gestión. |
| Cobertura pensada     | Positivos, negativos, bordes y validaciones de campos, no solo el camino feliz.                                                                                       |
| Ahorro de tokens      | Lo que se repite se automatiza una vez y después corre sin agentes.                                                                                                   |
| Seguridad             | Los secretos viven en el <code>.env</code> de la raíz, que nunca se versiona; los agentes nunca piden tokens por el chat.                                             |



Arquitectura de k0lmenIA · hacé clic para ver en tamaño completo

| PIEZA                  | QUÉ ES                                                                                                                        | DÓNDE                                            |
|------------------------|-------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------|
| Claude Code            | Orquestador: interpreta el pedido, elige el agente y aplica los estándares del proyecto.                                      | <code>CLAUDE.md</code>                           |
| Agentes                | El "quién": un especialista por tarea.                                                                                        | <code>.claude/agents/</code>                     |
| Skills                 | El "cómo": conocimiento que se carga cuando hace falta (técnicas de diseño, ejecución E2E y de API, convenciones de k0lmena). | <code>.claude/skills/</code>                     |
| Plantillas y scripts   | Formato de salida (planilla de casos, cobertura, bug) y reportes HTML determinísticos.                                        | <code>plantillas/</code> · <code>scripts/</code> |
| Integración de gestión | <code>gestion.py</code> con un adaptador por herramienta: Xray Cloud, Xray Server/DC, QTM4J y AIO Tests.                      | <code>scripts/gestion/</code>                    |
| Conectores MCP         | Playwright (activo), Jira y Confluence, Figma, Appium, QMetry y AIO Tests.                                                    | <code>.mcp.json</code>                           |
| Herramientas           | Newman para colecciones de Postman y k0lmena para web, API, mobile y performance.                                             | <code>herramientas/</code>                       |
| Entrada y salida       | Insumos del usuario y artefactos generados, ordenados por tipo.                                                               | <code>input/</code> · <code>output/</code>       |



1

**Poné tus insumos**

En `input/`, o pasá una key de Jira, una página de Confluence o un link de Figma con el conector activo.

2

**Pedí en lenguaje natural**

Claude Code elige el agente. También podés nombrarlo: *"usá el web-mapper para..."*.

3

**Revisá el resultado**

En `output/` o en `herramientas/k0lmena/`.

4

**Automatizá lo que se repite**

Y corrélo con `npm test` todas las veces que quieras, sin tokens.

| # | LE PEDÍS                                                                                     | OBTENÉS                                    |
|---|----------------------------------------------------------------------------------------------|--------------------------------------------|
| 1 | "Analizá la historia HU-001"                                                                 | Ambigüedades y preguntas para el PO        |
| 2 | "Armá el plan de pruebas de HU-001"                                                          | Plan HTML                                  |
| 3 | "Generá los casos de HU-001"                                                                 | <code>casos-HU-001.xlsx</code> + cobertura |
| 4 | "Subí los casos a Xray en 'HU-001 Registro', vinculados a PROJ-12, y creá el ciclo Sprint 5" | Casos y ciclo en Xray                      |
| 5 | "Automatizá en k0lmena los casos de HU-001 contra https://tu-app.com"                        | <code>.feature</code> + steps + locators   |
| 6 | <code>npm test</code> · <code>npm run report:web</code>                                      | Reporte HTML con evidencias                |
| 7 | "Subí los resultados al ciclo del Sprint 5"                                                  | Estados y evidencias en Xray               |
| 8 | "Armá una prueba de carga del login para 20 usuarios"                                        | Script k6/Artillery + reporte HTML         |
| 9 | "Armá el informe de cierre de HU-001"                                                        | Go / no-go                                 |

INTRODUCCIÓN

# Agentes

Claude Code elige el agente según lo que pidas. Están en `.claude/agents/` y se agrupan en cuatro familias.

## Análisis y diseño

**analista-historias**

Ambigüedades, vacíos, riesgos y preguntas de refinamiento de una historia.

`output/analisis-historias/`

**estratega-pruebas**

Plan de pruebas: alcance, riesgos, tipos de prueba, entorno y criterios, en HTML.

`output/planes-de-prueba/`

**generador-casos-manuales**

Casos en Excel y Markdown + informe de cobertura con preguntas para el PO.

`output/casos-de-prueba/manuales/`

**generador-casos-bdd**

Escenarios Gherkin (keywords en inglés, contenido en español) + cobertura.

`output/casos-de-prueba/bdd/`

**generador-casos-api**

Casos de API positivos, negativos, de schema y de autorización.

`output/casos-api/`

**generador-datos-prueba**

Datos válidos, inválidos y de borde en Markdown o CSV.

`output/datos-de-prueba/`

**generador-reportes-bug**

Reporte de bug profesional según la plantilla, con severidad y prioridad justificadas.

`output/reportes-bug/`

## Ejecución en vivo USA TOKENS EN CADA CORRIDA

**ejecutor-e2e**

Ejecuta casos en un navegador real con Playwright MCP (headed o headless), con evidencia.

`output/ejecuciones/`

**ejecutor-api**

Corre una colección de Postman con Newman.

`output/ejecuciones/`

**generador-reporte-html**

Dashboard HTML de los resultados de una ejecución.

`output/ejecuciones/`

**generador-reporte-cierre**

Informe de cierre de la ronda con recomendación go/no-go.

`output/informes-cierre/`

## Automatización con k0lmena DESPUÉS CORRE SIN TOKENS

**web-mapper**

Recorre la web siguiendo tus casos y genera `.feature`, steps y locators.

`k0lmena/web/`

**api-mapper**

Lee Postman o Swagger/OpenAPI, verifica los endpoints y genera los `.feature`.

`k0lmena/api/`

**mobile-mapper**

Recorre la app con Appium y genera `.feature`, steps y locators.

`k0lmena/mobile/`

**performance-mapper**

Carga, estrés, soak y picos con k6 o Artillery; pregunta carga y umbrales.

`k0lmena/performance/`



## Gestión de pruebas

### gestor-pruebas

Carpetas, casos, vínculos con la historia y ciclos en Xray, QMetry (QTM4J) o AIO Tests.

La herramienta +

`output/gestion/`

### publicador-resultados

Sube los resultados de

`npm test` al ciclo:

estado, error y evidencias.

La herramienta

# Requisitos

Instalá solo lo que vayas a usar.

| PARA...                         | NECESITÁS                                                                                                                   |
|---------------------------------|-----------------------------------------------------------------------------------------------------------------------------|
| Usar los agentes                | <b>Claude Code</b> y una cuenta de Claude (Pro, Max, Team o Enterprise) o acceso por API de Anthropic. VS Code recomendado. |
| Casos, reportes y gestión       | <b>Python 3</b> con <code>openpyxl</code> , <code>tabulate</code> y <code>requests</code> .                                 |
| Ejecutar E2E en vivo            | <b>Node.js 18+</b> y los navegadores de Playwright.                                                                         |
| Ejecutar colecciones de Postman | <b>Newman</b> .                                                                                                             |
| Automatizar con k0lmena         | <b>Node.js 20+</b> .                                                                                                        |
| Automatizar mobile              | <b>Node.js 22+</b> , JDK y Android SDK, o macOS con Xcode para iOS; o una cuenta de BrowserStack.                           |

# Instalación

En cinco minutos tenés los agentes andando; k0lmena y el resto se suman cuando los necesites.

1

## Instalá Claude Code

```
curl -fsSL https://claude.ai/install.sh | bash      # macOS / Linux
npm install -g @anthropic-ai/claude-code           # alternativa (Node.js 18+)
claude --version
```

En Windows, seguí la [guía oficial](#).

2

## Cloná el repositorio

```
git clone https://github.com/underc0delabs/k0lmenIA.git
cd k0lmenIA
pip install -r requirements.txt
cp .env.example .env                                # y completalo
```

3

## Abrí VS Code y lanzá Claude Code

```
code .
claude                                             # la primera vez pide autenticarte
```

4

## Sumá lo opcional

```
npx playwright install                        # ejecución E2E en vivo
npm install -g newman                        # colecciones de Postman
cd herramientas/k0lmena
npm install                                  # k0lmena
npx playwright install chromium
npm run bootstrap:k6                          # performance de APIs con k6
```

PRIMEROS PASOS

# Configuración: el archivo .env

Hay **un solo** `.env`, **en la raíz del repo**. Lo usan los agentes, k0lmena y los scripts de gestión. Está en `.gitignore` y la plantilla comentada es `.env.example`.

| GRUPO           | VARIABLES                                                                                                                                                               | PARA QUÉ                         |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------|
| App bajo prueba | <code>APP_URL</code> <code>APP_USER</code> <code>APP_PASSWORD</code>                                                                                                    | Ejecución E2E y logins           |
| API             | <code>API_TOKEN</code> <code>API_BASEURL</code>                                                                                                                         | Newman, suite de API y k6        |
| Web             | <code>BASEURL</code> <code>BROWSER</code> <code>HEADLESS</code> <code>VIEWPORT_WIDTH</code><br><code>VIEWPORT_HEIGHT</code> <code>LOCALE</code> <code>TIMEZONE</code>   | Navegador de la suite web        |
| Ejecución       | <code>TAGS</code> <code>PARALLEL</code>                                                                                                                                 | Qué escenarios y cómo se corren  |
| Evidencias      | <code>EVIDENCE</code> ( <code>captura</code> · <code>ambos</code> · <code>off</code> ) <code>VIDEO</code> <code>TRACE</code>                                            | Qué se adjunta al reporte        |
| Auto-healing    | <code>KOLMENA_AUTO_HEALING</code>                                                                                                                                       | Recuperar locators rotos         |
| Mobile          | <code>MOBILE_TARGET</code> <code>MOBILE_PLATFORM</code> <code>MOBILE_DEVICE_NAME</code><br><code>MOBILE_APP</code> <code>MOBILE_UDID</code> <code>BROWSERSTACK_*</code> | Dónde corre la suite mobile      |
| Performance     | <code>PERF_VUS</code> <code>PERF_DURACION</code>                                                                                                                        | Pisar la carga de los scripts k6 |
| Gestión         | <code>GESTION_HERRAMIENTA</code> <code>GESTION_PROYECTO</code> + credenciales de la herramienta                                                                         | Xray, QMetry y AIO Tests         |

! Usá credenciales de un entorno de prueba, nunca de producción. Los agentes leen los secretos del `.env`: nunca pegues un token en el chat ni lo escribas en un archivo versionado.

## Carpetas de entrada

| CARPETA                           | QUÉ VA                                                                                |
|-----------------------------------|---------------------------------------------------------------------------------------|
| <code>input/historias/</code>     | Historias de usuario con sus criterios de aceptación ( <code>HU-001.md</code> , ...). |
| <code>input/documentacion/</code> | Documentación del producto que sirva de contexto.                                     |
| <code>input/api/</code>           | Contratos, colecciones de Postman y environments.                                     |
| <code>input/bugs/</code>          | Observaciones sueltas de errores para convertir en reportes.                          |

# Diseño y ejecución

Qué pedir, qué hace cada agente y qué deja.

## Analizar una historia

ANALISTA-HISTORIAS

**PEDIDO** · Analizá la historia HU-001 y decime qué ambigüedades tiene.

Revisa cada criterio y entrega `analisis-HU-001.md` con ambigüedades, vacíos, riesgos, casos no contemplados y preguntas para el PO.

## Armar el plan de pruebas

ESTRATEGA-PRUEBAS

**PEDIDO** · Armá el plan de pruebas de HU-001.

Dashboard HTML con alcance, objetivos, tipos de prueba, riesgos, datos y entorno, y criterios de entrada y salida.

## Generar casos manuales

GENERADOR-CASOS-MANUALES

**PEDIDO** · Generá los casos de prueba de HU-001.

Entrega `casos-HU-001.xlsx`, `casos-HU-001.md` y `casos-HU-001-cobertura.md`. Cubre camino feliz, negativos, bordes y validaciones. Esa planilla es la que después se sube a la herramienta de gestión o se automatiza.

## Generar escenarios BDD

GENERADOR-CASOS-BDD

**PEDIDO** · Pasá la historia HU-001 a escenarios BDD.

Genera `HU-001-<slug>.feature`, con un tag `@CP-XXX` por escenario, y `HU-001-cobertura.md`.

## Generar casos de API

GENERADOR-CASOS-API

**PEDIDO** · Generá los casos de la API de autenticación a partir de `input/api/auth-endpoints.md`.

Tabla resumen y detalle por caso: método, endpoint, headers, body, status y schema esperado.

## Datos de prueba y bugs

GENERADOR-DATOS-PRUEBA

GENERADOR-REPORTES-BUG

**PEDIDO** · Generá datos de prueba para el formulario de registro, en CSV.

Datos válidos, inválidos y de borde. Los bugs siguen `plantillas/plantilla-reporte-bug.md` (editala para usar tu formato) y marcan lo que falta en vez de inventarlo.

## Ejecutar en vivo

[EJECUTOR-E2E](#)[EJECUTOR-API](#)[USA TOKENS](#)

**PEDIDO** · Ejecutá *SOLO* el escenario de registro válido de HU-001 contra *https://tu-app.com*.

El ejecutor E2E pregunta si querés ver el navegador, ejecuta con Playwright MCP, saca evidencia y genera `reporte-HU-001-<fecha-hora>.html`. El de API corre la colección con Newman. Ideal para pruebas puntuales: lo que se repite conviene automatizarlo.

## Reportes e informe de cierre

[GENERADOR-REPORTE-HTML](#)[GENERADOR-REPORTE-CIERRE](#)

**PEDIDO** · Armá el informe de cierre de las pruebas de HU-001.

Dashboards con indicadores, gráficos y detalle; el informe de cierre suma bugs abiertos, riesgos y la recomendación go/no-go.

# Automatización con k0lmena

El framework integrado en `herramientas/k0lmena/`. Un agente *mapper* recorre la aplicación **una sola vez** y escribe la automatización; desde ahí la suite corre con `npm`, sin agentes y sin tokens.



## Web

Playwright + Cucumber, video, trace y GIF. Con `K0LMENA_AUTO_HEALING=on` recupera locators rotos.



## API

axios + Cucumber con steps genéricos en español: no hace falta programar.



## Mobile

WebdriverIO + Appium en dispositivo, emulador o BrowserStack.



## Performance

k6 para APIs y Artillery + Playwright para flujos web.

- i Las carpetas de k0lmena vienen vacías: todo lo que hay adentro lo generan los mappers para tu aplicación.

## web-mapper

**PEDIDO** · Automatizá en k0lmena los casos de output/casos-de-prueba/manuales/casos-HU-001.xlsx contra `https://tu-app.com`.

1. Lee los casos (manuales o BDD).
2. Navega la web real con Playwright MCP y comprueba que cada paso se puede ejecutar.
3. Genera `web/features/HU-001-<slug>.feature`, `web/steps/HU-001.steps.ts` y `web/locators/HU-001.locators.ts`, reutilizando steps existentes.
4. Valida con `npm run test:web` y deja `output/mapeos/mapeo-HU-001-web.md`.

Si un paso no se puede ejecutar, el escenario queda `@bloqueado` con el motivo y no corre hasta que se resuelva.

## api-mapper

**PEDIDO** · Pasá a k0lmena los endpoints de /pet del Swagger `https://petstore.swagger.io/v2/swagger.json`.

Lee Postman (v2.x) o Swagger/OpenAPI, te deja elegir los endpoints, diseña escenarios positivos, negativos y de autorización, los verifica contra la API real y escribe los `.feature` con los steps genéricos. Antes de un POST, PUT, PATCH o DELETE pide autorización.

## mobile-mapper

**PEDIDO** · Automatizá en k0lmena los casos de login de la app en el emulador.

Necesita el conector Appium MCP y la app en `mobile/apps/` (no se versiona). Después la suite corre sin MCP según `MOBILE_TARGET`:

| DESTINO                   | QUÉ NECESITÁS                                                                                    |
|---------------------------|--------------------------------------------------------------------------------------------------|
| <code>device</code>       | Dispositivo por USB con depuración y <code>MOBILE_UDID</code> (de <code>adb devices</code> ).    |
| <code>emulator</code>     | Emulador o simulador encendido y <code>MOBILE_DEVICE_NAME</code> .                               |
| <code>browserstack</code> | <code>BROWSERSTACK_USER</code> , <code>BROWSERSTACK_KEY</code> y <code>BROWSERSTACK_APP</code> . |

## Correr la suite

```
cd herramientas/k0lmena
npm test # web + API
npm run test:web # también: test:api · test:mobile · test:all
TAGS=@HU-001 npm test # filtrar por tag (bash)
$env:TAGS="@HU-001"; npm test # filtrar por tag (PowerShell)
npm run report:web # también: report:api · report:mobile
```

Si `EVIDENCE` está vacía, `npm test` pregunta al arrancar qué guardar de los tests que pasan: solo captura, o captura + GIF del recorrido.

## Evidencias en el reporte

| SUITE  | ESCENARIO QUE PASA                                           | ESCENARIO QUE FALLA                                                                                                                                                      |
|--------|--------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Web    | Captura final (+ GIF opcional)                               | Video completo, captura, error con stack, URL y título, logs del navegador (consola, errores JS, requests fallidos, HTTP $\geq$ 400), logs de Node y trace de Playwright |
| API    | Request y response (el <code>Authorization</code> se oculta) | Lo mismo, más la validación que falló                                                                                                                                    |
| Mobile | Captura final (+ GIF opcional)                               | Video completo, captura, page source y logs del dispositivo                                                                                                              |

## Herramientas extra

| COMANDO                            | PARA QUÉ                                                 |
|------------------------------------|----------------------------------------------------------|
| <code>npm run crawler [url]</code> | Genera un archivo de locators recorriendo una página.    |
| <code>npm run link-tester</code>   | Busca enlaces e imágenes rotas en <code>BASEURL</code> . |
| <code>npm run record</code>        | Graba un flujo con Playwright codegen.                   |
| <code>npm run debug</code>         | Corre la suite web en modo debug.                        |



# Pruebas de performance

El **performance-mapper** arma pruebas de carga, estrés, soak y picos. Nunca usa valores por defecto: la carga y los umbrales los definís vos.

! Una prueba de carga genera tráfico real. Corredla solo contra ambientes de prueba o con autorización de los dueños del sistema.

## performance-mapper

**PEDIDO** · Armá una prueba de carga de los endpoints de /pet para 50 usuarios.

1. **Pregunta** el ambiente (y que no sea producción), el flujo o los endpoints, la carga objetivo, los umbrales (p95, p99, % de error), los perfiles y si puede usar métodos con efecto.
2. **Elige la herramienta**: k6 para APIs (escala a miles de usuarios) o Artillery + Playwright para flujos de navegador (pocos usuarios: cada uno es un Chromium).
3. **Escribe el script** una vez y lo valida con un smoke.
4. **Corre con carga** solo con tu confirmación explícita para cada corrida.

## Comandos

```
npm run perf # lista los scripts
npm run perf -- <script> smoke # valida el script con carga mínima
npm run perf -- <script> load # carga objetivo (pide confirmación)
npm run perf -- <script> stress --confirmar # sin pregunta: CI o con autorización
npm run perf -- <script> load --vus 20 --duracion 2m # k6: pisa la carga
```

| PERFIL | QUÉ HACE                                                             |
|--------|----------------------------------------------------------------------|
| smoke  | Carga mínima para validar que el script funciona.                    |
| load   | Sube hasta la carga objetivo, la sostiene y baja.                    |
| stress | 1x, 2x y 3x la carga objetivo: busca el punto de quiebre.            |
| soak   | Carga objetivo durante mucho tiempo: fugas de memoria y degradación. |
| spike  | Salto brusco a un pico y vuelta: cómo se recupera.                   |

Cada corrida deja en `reports/performance/<k6|artillery>/` un reporte HTML con el veredicto, requests, req/s, errores, latencias p50/p95/p99, umbrales con el valor medido, detalle por endpoint o paso, códigos de respuesta, checks y, en Artillery, la evolución en el tiempo. Si un umbral no se cumple, el comando termina con error.

# Xray, QMetry y AIO Tests

**gestor-pruebas** y **publicador-resultados** trabajan con Xray Cloud, Xray Server/Data Center, QMetry para Jira (QTM4J) y AIO Tests, con los mismos comandos para todas.

Se configura en el `.env`: `GESTION_HERRAMIENTA` ( `xray-cloud`, `xray-dc`, `qtm4j` o `aio` ), `GESTION_PROYECTO` (la key del proyecto de Jira) y las credenciales de tu herramienta, explicadas en `.env.example`.

## gestor-pruebas

**PEDIDO** · Subí los casos de HU-001 a Xray en la carpeta 'HU-001 Registro', vinculados a PROJ-12, y creá el ciclo Sprint 5.

Crea la carpeta, sube los casos (manuales desde el `.xlsx` o BDD desde el `.feature`), los vincula a la historia, crea ciclos y les agrega casos. La primera vez hace una prueba en seco ( `--dry-run` ). Guarda la trazabilidad en `output/gestion/HU-001-<herramienta>.json`: un caso nunca se sube dos veces.

## publicador-resultados

**PEDIDO** · Subí los resultados de la última corrida web al ciclo PROJ-60.

Asocia cada escenario a su caso por el tag `@CP-XXX` y carga el estado, un comentario con el error y las evidencias: captura siempre, GIF si se generó y video solo si falló. Nunca asigna un resultado por aproximación.

## Equivalencias

| CONCEPTO         | XRAY                           | QTM4J            | AIO TESTS            |
|------------------|--------------------------------|------------------|----------------------|
| Caso             | Issue Test (Manual o Cucumber) | Test case        | Caso (Classic o BDD) |
| Ciclo            | Test Execution                 | Test cycle       | Test cycle           |
| Caso ↔ historia  | Link "Test" (cobertura)        | Requirement link | jiraRequirementIDs   |
| Ciclo ↔ historia | Link "Relates"                 | Requirement link | jiraTaskIDs          |

## Comandos manuales

```
python scripts/gestion/gestion.py --dry-run carpeta --ruta "HU-001 Registro"
python scripts/gestion/gestion.py subir-casos --origen <casos.xlsx> --carpeta "HU-001 Registro" --historia PROJ-12 --traza HU-001
python scripts/gestion/gestion.py crear-ciclo --nombre "Sprint 5" --historia PROJ-12 --traza HU-001
python scripts/gestion/gestion.py extraer-resultados --reporte <cucumber-report.json>
python scripts/gestion/gestion.py publicar-resultados --ciclo PROJ-60 --resultados <resultados.json> --traza HU-001
```

Las carpetas van sin barra inicial (en Git Bash, `/HU-001` se convierte en una ruta de Windows). Referencia completa en `scripts/gestion/README.md`.

# Conectores MCP

Conectan a Claude Code con sistemas externos. `.mcp.json` trae activo Playwright; el resto está en `.mcp.json.example`.

| CONECTOR                                 | PARA QUÉ                                                                                                | AUTENTICACIÓN                |
|------------------------------------------|---------------------------------------------------------------------------------------------------------|------------------------------|
| <code>playwright</code>                  | Navegador para el ejecutor E2E y el web-mapper (headed y headless).                                     | Activo                       |
| <code>atlassian</code>                   | Jira y Confluence: historias, criterios y documentación funcional directamente.                         | Cuenta (OAuth)               |
| <code>figma</code>                       | Diseños desde un link a un frame o archivo: textos, estados y componentes. Requiere asiento Full o Dev. | Cuenta (OAuth)               |
| <code>appium-mcp</code>                  | Lo usa el mobile-mapper para recorrer la app.                                                           | <code>ANDROID_ID_HOME</code> |
| <code>qmetry</code> · <code>qtm4j</code> | Consultar QMetry desde el chat.                                                                         | API key                      |
| <code>aio-tests</code>                   | Consultar AIO Tests desde el chat.                                                                      | Token                        |

- 1

Copíala entrada

De `.mcp.json.example` a `.mcp.json`.
- 2

Habilitala

Agregá su nombre a `enabledMcpjsonServers` en `.claude/settings.local.json`.
- 3

Autentica

Si usa token, definilo en el entorno antes de lanzar `claude`. Atlassian y Figma se autorizan con tu cuenta desde `/mcp`.
- 4

Verificá

Reiniciá Claude Code y revisá la conexión con `/mcp`.

**PEDIDO** · *Generá los casos de PROJ-12 usando la página de Confluence 'Reglas de facturación' y este diseño de Figma.*

Para Xray no hace falta MCP: se usa `scripts/gestion/`. Guía completa en `CONECTORES.md`.

# Convenciones

| ELEMENTO                | FORMATO    |
|-------------------------|------------|
| Historias               | HU-001     |
| Criterios de aceptación | CA1 · CA2  |
| Casos manuales          | CP-001     |
| Casos de API            | CP-API-001 |
| Bugs                    | BUG-001    |

| ESCALA                | VALORES                                                         |
|-----------------------|-----------------------------------------------------------------|
| Severidad y prioridad | Crítica · Alta · Media · Baja                                   |
| Estado de un caso     | N/A · Pendiente · En ejecución · Aprobado · Fallido · Bloqueado |

## Tags en k0lmena

| DÓNDE    | TAGS                                                                   |
|----------|------------------------------------------------------------------------|
| Feature  | La historia y el tipo: @HU-001 @web · @api · @mobile                   |
| Scenario | El ID del caso ( @CP-001 , @CP-API-001 ) y @Smoke si es crítico        |
| Excluir  | @bloqueado : un paso no se pudo verificar y el escenario no se ejecuta |

## Nombres de archivos

| ARTEFACTO      | NOMBRE                                                                                      |
|----------------|---------------------------------------------------------------------------------------------|
| Análisis       | analisis-HU-001.md                                                                          |
| Casos manuales | casos-HU-001.xlsx + .md + -cobertura.md                                                     |
| Casos BDD      | HU-001-registro.feature + HU-001-cobertura.md                                               |
| Bug            | BUG-001.md                                                                                  |
| Automatización | <tipo>/features/HU-001-<slug>.feature · steps/HU-001.steps.ts · locators/HU-001.locators.ts |
| Performance    | performance/k6/http/HU-001-<slug>.ts · performance/artillery/HU-001-<slug>.yaml             |
| Mapeo          | output/mapeos/mapeo-HU-001-<tipo>.md                                                        |

# Estructura del repositorio

```

k0lmenIA/
├─ CLAUDE.md           Contexto y estándares (Claude Code lo lee siempre)
├─ ARQUITECTURA.md    Cómo crece el repo
├─ CONECTORES.md       Cómo activar los conectores MCP
├─ .env.example        Plantilla de variables (copiar a .env)
├─ .mcp.json           Conexiones MCP activas (Playwright)
├─ .mcp.json.example   Conectores opcionales
├─ docs/              Este sitio, el PDF y los diagramas
├─ .claude/
│   └─ agents/         Los 17 agentes
│   └─ skills/         Diseño, ejecución E2E y API, k0lmena
├─ input/             historias/ documentacion/ api/ bugs/
├─ output/            Lo que generan los agentes
├─ plantillas/        Bug, planilla de casos, cobertura
├─ scripts/           Casos, reportes HTML, plan, informe de cierre
│   └─ gestion/        Xray, QMetry (QTM4J) y AIO Tests
├─ herramientas/
│   └─ newman/         Colecciones de Postman
│   └─ k0lmena/
│       └─ web/        features/ steps/ locators/ + framework
│       └─ api/        features/ steps/ (steps genéricos)
│       └─ mobile/     features/ steps/ locators/ support/ apps/
│       └─ performance/ k6/http/ y artillery/
│       └─ reports/    Reportes HTML
│       └─ run-tests.js Runner de npm test
│       └─ run-perf.js  Runner de npm run perf

```

## Problemas frecuentes

Un agente nuevo no aparece

La suite web falla con un viewport de 0x0 o sin URL

`npm test`

corre 0 escenarios

Un escenario nunca se ejecuta

"No encuentro k6"

`npm run perf`

no corre un perfil con carga desde el agente

Una carpeta de Xray/QMetry/AIO aparece como

`C:/Program Files/Git/...`

Un conector MCP no conecta

Falta una librería de Python

# Extender el proyecto

## Agregar un agente

- 1

**Creá el archivo**  
`.claude/agents/mi-agente.md` con frontmatter `name` y `description`. La descripción es lo que usa Claude Code para decidir cuándo invocarlo.
- 2

**Escribí el cuerpo**  
En español: rol, entradas, proceso, salida y reglas.
- 3

**Definí el formato**  
Si genera un formato propio, sumá su plantilla en `plantillas/` o su script en `scripts/`, y su carpeta en `output/`.
- 4

**Normalizá las tablas**  
Si genera tablas en un `.md`, pasalas por `scripts/formatear_tablas.py`.

## Skills, conectores y herramientas

**Skill**  
Una carpeta en `.claude/skills/` con su `SKILL.md`; se carga solo cuando hace falta.

**Conector MCP**  
Una entrada en `.mcp.json`; los tokens siempre como `${VARIABLE}`.

**Herramienta**  
Una subcarpeta en `herramientas/` con su README.

## Regenerar esta documentación

```
node docs/generar.js      # diagramas (PNG) y PDF, con el Playwright de k0lmena
```

El sitio es `docs/index.html` (se puede publicar con GitHub Pages desde la carpeta `docs/`); el PDF sale del mismo HTML con estilos de impresión, y los diagramas de `docs/diagramas/`.

## Licencia

Licencia **MIT** — © 2026 **QARMY**. k0lmena es un proyecto open source de **Danilo Vezzoni**. Hecho por la comunidad de **Underc0de**.